Consolidated Sampling code

BOIKARABELO OLIFANT

2026-10-30

```
suppressPackageStartupMessages({
  library(tidyverse)
  library(survey)
})
```

FINITE POPULATION SIMULATION (N = 10000)

From the Adrian Frich census 0f 2011, we used the metropolis Ekurhuleni, and the residential information of Boksburg as the reference for our proportions for the different residential areas. That is we shall start with proportions of the total population as townships (50%), mid-income (28%), and high-income (22%). These numbers may vary in current reality, but they are a safe estimate for the proportions of total residents in a study area.

```
set.seed(1852)

N <- 10000
size_low  <- .5*N
size_mid  <- .28*N
size_high <- .22*N



c(size_low, size_mid, size_high)
```

```
## [1] 5000 2800 2200
```

```
sum(c(size_low, size_mid, size_high))
```

```
## [1] 10000
```

```
neighborhoods <- c(
  rep("Low-Income (Informal/Township)", size_low),
  rep("Mid-Income Suburb", size_mid),
  rep("High-Income Suburb", size_high)
)

incomes_low  <- runif(size_low,  min = 5000,  max = 15000)
incomes_mid  <- runif(size_mid,  min = 18000, max = 35000)
```

```r
incomes_high <- runif(size_high, min = 45000, max = 75000)

all_incomes <- c(incomes_low, incomes_mid, incomes_high)

pop <- tibble(
  id = 1:N,
  strata = neighborhoods,
  Y = all_incomes
)

head(pop)
```

```
## # A tibble: 6 x 3
##      id strata                              Y
##   <int> <chr>                           <dbl>
## 1     1 Low-Income (Informal/Township)  7944.
## 2     2 Low-Income (Informal/Township)  7218.
## 3     3 Low-Income (Informal/Township)  9808.
## 4     4 Low-Income (Informal/Township) 12473.
## 5     5 Low-Income (Informal/Township)  5097.
## 6     6 Low-Income (Informal/Township)  5675.
```

```r
tail(pop)
```

```
## # A tibble: 6 x 3
##      id strata                    Y
##   <int> <chr>                 <dbl>
## 1  9995 High-Income Suburb 50583.
## 2  9996 High-Income Suburb 74219.
## 3  9997 High-Income Suburb 58132.
## 4  9998 High-Income Suburb 59469.
## 5  9999 High-Income Suburb 50772.
## 6 10000 High-Income Suburb 54616.
```

```r
# Extract True Baseline Parameters using exact course rounding notation
Y_bar_pop <- sum(pop$Y) / length(pop$Y)
Y_bar_pop <- as.numeric(format(round(Y_bar_pop, 1)))

S2_pop <- sum((pop$Y - Y_bar_pop)^2) / (length(pop$Y) - 1)
S2_pop <- as.numeric(format(round(S2_pop, 2)))

print(paste("True Population Mean:", Y_bar_pop))
```

```
## [1] "True Population Mean: 25688.4"
```

```r
print(paste("True Population Variance (S^2):", S2_pop))
```

```
## [1] "True Population Variance (S^2): 413934008"
```

SAMPLE SIZE ESTIMATION AND SENSITIVITY PLOT

```
# Define an error range vector for the x-axis mapping
Required_error <- seq(100, 2000, by = 10)
error <- Required_error

# Compute Cochran sample size curves across the four parameter scales
sample_size_100 <- ((1.96 * sqrt(S2_pop)))^2 / (error)^2
sample_size_100 <- sample_size_100 / (1 + (1/N) * sample_size_100)

sample_size_75  <- ((1.96 * sqrt(S2_pop * 0.75)))^2 / (error)^2
sample_size_75  <- sample_size_75 / (1 + (1/N) * sample_size_75)

sample_size_50  <- ((1.96 * sqrt(S2_pop * 0.50)))^2 / (error)^2
sample_size_50  <- sample_size_50 / (1 + (1/N) * sample_size_50)

sample_size_25  <- ((1.96 * sqrt(S2_pop * 0.25)))^2 / (error)^2
sample_size_25  <- sample_size_25 / (1 + (1/N) * sample_size_25)
```

Sample size plot

```
pdf("Sample size plot.pdf", width= 6, height=5)

plot(Required_error, sample_size_100, type="l", xlab = "Margin of error (Rands)",
     ylab = "Sample size", yaxt='n', lwd=3, xlim=c(100, 2000), ylim=c(0, 1500),
     main="Sample Size Estimates by True Population Variance")
par(new=TRUE)
plot(Required_error, sample_size_75, type="l", xlab = "", ylab = "", xaxt='n', yaxt='n',
     col="green", lwd=3, xlim=c(100, 2000), ylim=c(0, 1500))
par(new=TRUE)
plot(Required_error, sample_size_50, type="l", xlab = "", ylab = "", xaxt='n', yaxt='n',
     col="blue", lwd=3, xlim=c(100, 2000), ylim=c(0, 1500))
par(new=TRUE)
plot(Required_error, sample_size_25, type="l", xlab = "", ylab = "", xaxt='n', yaxt='n',
     col="purple", lwd=3, xlim=c(100, 2000), ylim=c(0, 1500))
axis(side = 4)

# Set our target 5% error line boundary threshold
e_target <- 0.05 * Y_bar_pop
n_target <- ((1.96 * sqrt(S2_pop)))^2 / (e_target)^2

k_interval <- round(N / n_target)


n_target <- N/k_interval

abline(v = e_target, lwd = 2, col = "darkorange")
abline(h = n_target, lwd = 2, col = "darkorange")
abline(h = N, lwd = 2, col = "red")

legend("topright", c("100% Baseline Variance", "75% Baseline Variance", "50% Baseline Variance", "25% Ba
```

```
       col = c("black", "green", "blue", "purple"), lty = 1, lwd = 2)

dev.off()
```

```
## pdf
##   2
```

```
e_target
```

```
## [1] 1284.42
```

```
n_target
```

```
## [1] 1000
```

```
k_interval
```

```
## [1] 10
```

```
n_target*k_interval
```

```
## [1] 10000
```

Calculated sample size is 1000 households for a 5% error of (R1284.42)

We also see that the N= nk, just like the notes suggest.

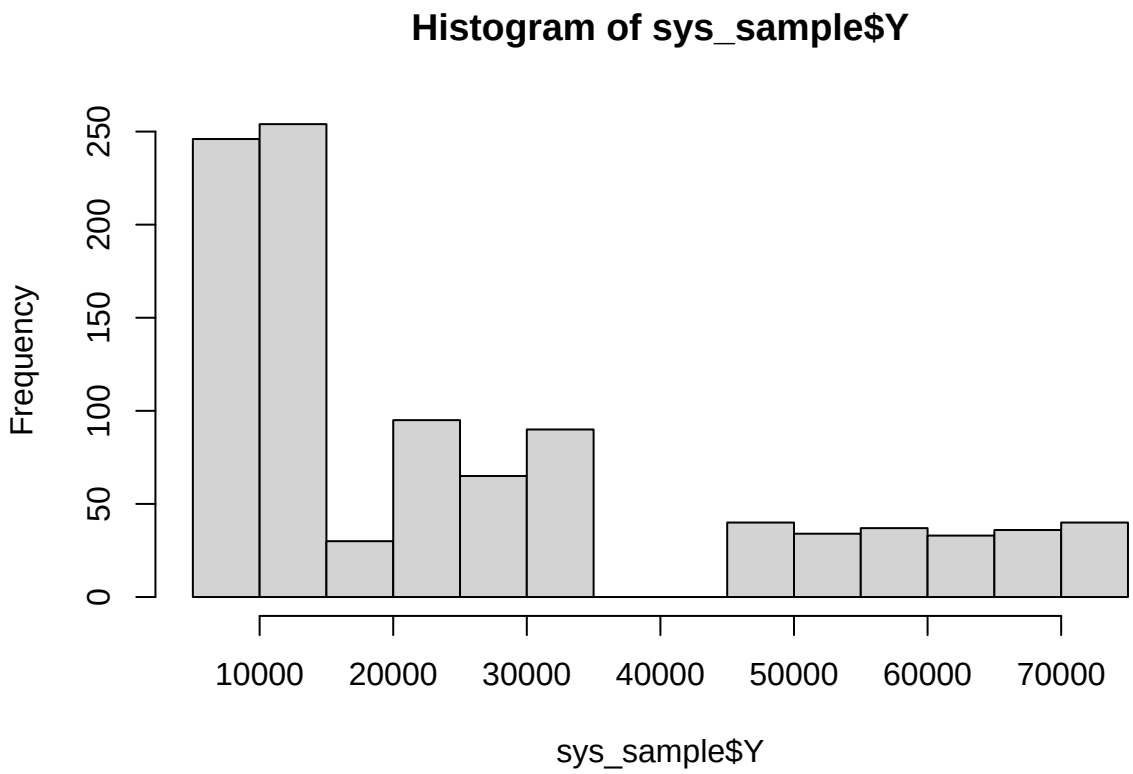
EXECUTE ONE REALIZED PROBABILITY SAMPLE SELECTION

```
set.seed(1852)
# Systematic selection execution
random_start <- sample(1:k_interval, size = 1)
sys_rows <- seq(from = random_start, to = N, by = k_interval)[1:n_target]
sys_sample   <- pop %>% filter(id %in% sys_rows) %>% mutate(w = N/n_target, fpc = N)


head(sys_sample)
```

```
## # A tibble: 6 x 5
##      id strata                              Y     w   fpc
##   <int> <chr>                           <dbl> <dbl> <dbl>
## 1     7 Low-Income (Informal/Township) 14057.    10 10000
## 2    17 Low-Income (Informal/Township)  7710.    10 10000
## 3    27 Low-Income (Informal/Township) 14542.    10 10000
## 4    37 Low-Income (Informal/Township)  6276.    10 10000
## 5    47 Low-Income (Informal/Township) 11548.    10 10000
## 6    57 Low-Income (Informal/Township) 13749.    10 10000
```

```r
hist(sys_sample$Y)
```

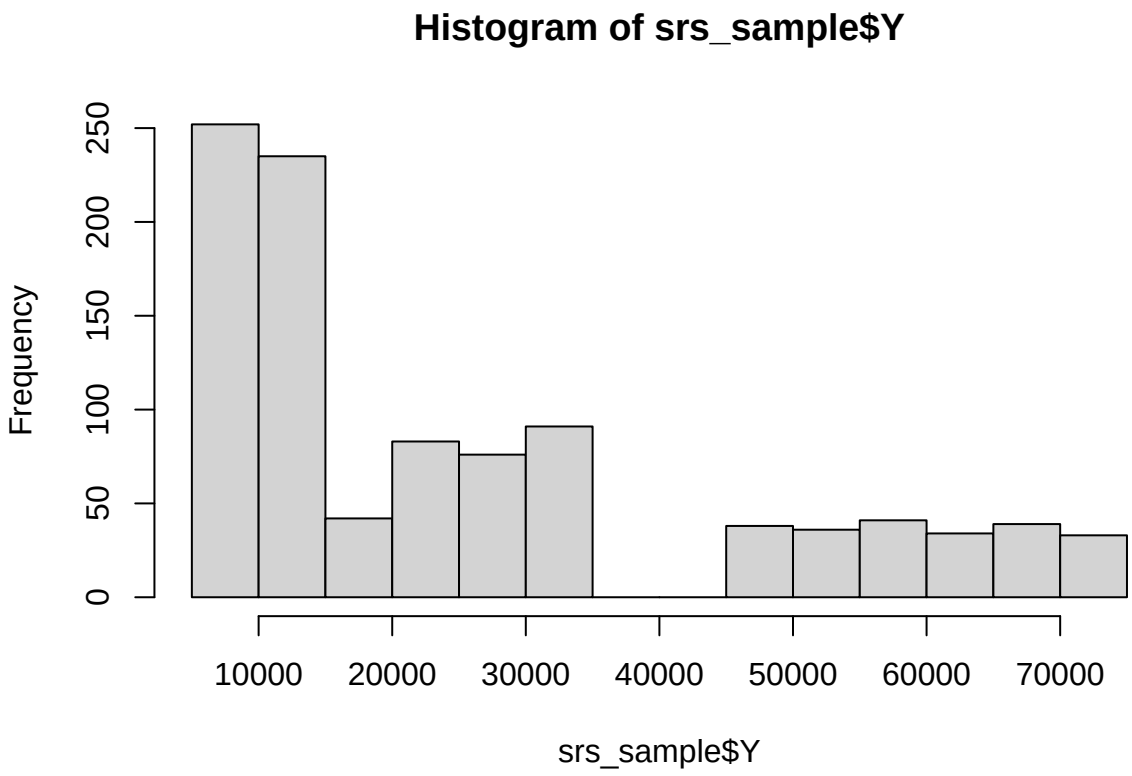



```r
# SRSWOR benchmark sample draw
srs_rows <- sample(pop$id, size = n_target, replace = FALSE)
srs_sample <- pop %>% filter(id %in% srs_rows) %>% mutate(w = N/n_target, fpc = N)

head(srs_sample)
```

```
## # A tibble: 6 x 5
##      id strata                              Y     w   fpc
##   <int> <chr>                           <dbl> <dbl> <dbl>
## 1     2 Low-Income (Informal/Township)  7218.    10 10000
## 2    17 Low-Income (Informal/Township)  7710.    10 10000
## 3    24 Low-Income (Informal/Township) 13013.    10 10000
## 4    29 Low-Income (Informal/Township)  7535.    10 10000
## 5    37 Low-Income (Informal/Township)  6276.    10 10000
## 6    54 Low-Income (Informal/Township) 10637.    10 10000
```

```r
hist(srs_sample$Y)
```



DESIGN-BASED INFERENCE (Manual Formulas vs Package Check)

```
# Manual Calculations (Treatment under SRSWOR Approximation rule)
y_bar_sys   <- mean(sys_sample$Y)
var_y_sys   <- sum((sys_sample$Y - y_bar_sys)^2) / (length(sys_sample$Y) - 1)
sys_se_man  <- sqrt((1 - n_target/N) * var_y_sys / n_target)

# Formal Package Verification via Survey Objects
sys_design <- svydesign(ids = ~1, weights = ~w, fpc = ~fpc, data = sys_sample)
sys_package_mean <- svymean(~Y, design = sys_design)


print(paste("Manual Sample Mean Estimate: R", round(y_bar_sys, 2)))
```

```
## [1] "Manual Sample Mean Estimate: R 25679.21"
```

```
print(paste("Manual Standard Error:       R",
round(sys_se_man, 2)))
```

```
## [1] "Manual Standard Error:       R 609.5"
```

```r
print(paste("Package Survey Sample Mean: R",
round(sys_package_mean, 2)))
```

```
## [1] "Package Survey Sample Mean: R 25679.21"
```

```r
print(paste("Package Survey SE Vector:    R", round(SE(sys_package_mean), 2)))
```

```
## [1] "Package Survey SE Vector:    R 609.5"
```

SAMPLING DISTRIBUTION SIMULATION

```r
B <- 1000

# Monte Carlo simulation using the native course replicate framework
sys_distribution_means <- replicate(B, {
  iter_start <- sample(1:k_interval, size = 1)
  iter_rows  <- seq(from = iter_start, to = N, by = k_interval)[1:n_target]
  mean(pop$Y[iter_rows])
})

srs_distribution_means <- replicate(B, {
  iter_ids <- sample(pop$id, size = n_target, replace = FALSE)
  mean(pop$Y[iter_ids])
})
```

EXPLORATORY SAMPLING DISTRIBUTION PLOTS

```r
true_mean <- Y_bar_pop


pdf("sample means.pdf", width=6, height = 5)
par(mfrow = c(1, 2))




# This ensures a fair visual comparison of the spread width
all_means <- c(sys_distribution_means, srs_distribution_means)
min_x <- min(all_means) - 100
max_x <- max(all_means) + 100

#  Systematic Sampling Distribution
hist(sys_distribution_means,
     main = "Systematic Sampling Distribution",
     xlim = c(min_x, max_x),
     xlab = "Sample Means (Rands)",
     ylab = "Number of Possible Samples",
     col = blues9,
     breaks = 25)
```

```r
# Vertical dashed line for the absolute true population mean
abline(v = true_mean, lwd = 3, col = "darkorange", lty = 2)

# SRSWOR Benchmark Distribution
hist(srs_distribution_means,
     main = "SRSWOR Benchmark Distribution",
     xlim = c(min_x, max_x),
     xlab = "Sample Means (Rands)",
     ylab = "Number of Possible Samples",
     col = blues9,
     breaks = 25)
# Add the same vertical reference line to show the truth location
abline(v = true_mean, lwd = 3, col = "darkorange", lty = 2)

# 5. Reset the plotting layout back to a single window framework
par(mfrow = c(1, 1))
dev.off()
```

```
## pdf
##   2
```

```r
  Systematic = c(mean(sys_distribution_means), sd(sys_distribution_means))
  SRSWOR = c(mean(srs_distribution_means), sd(srs_distribution_means))
  True_Population_Value = c(Y_bar_pop, sqrt((1 - n_target/N) * S2_pop / n_target))

Systematic
```

```
## [1] 25683.7297   167.0744
```

```r
SRSWOR
```

```
## [1] 25667.3451   609.0745
```

```r
True_Population_Value
```

```
## [1] 25688.400   610.361
```

CI EXTRACTION LOOP & PERFORMANCE SCORES

```r
set.seed(1852)

# 1. Running  loops t capture the standard errors for each iteration
sys_distribution_se <- replicate(B, {
  iter_start <- sample(1:k_interval, size = 1)
  iter_rows  <- seq(from = iter_start, to = N, by = k_interval)[1:n_target]
  iter_s2    <- sum((pop$Y[iter_rows] - mean(pop$Y[iter_rows]))^2) / (n_target - 1)
  sqrt((1 - n_target/N) * iter_s2 / n_target)
```

```
})



srs_distribution_se <- replicate(B, {
  iter_ids   <- sample(pop$id, size = n_target, replace = FALSE)
  iter_s2    <- sum((pop$Y[iter_ids] - mean(pop$Y[iter_ids]))^2) / (n_target - 1)
  sqrt((1 - n_target/N) * iter_s2 / n_target)
})
```

Derive the confidence interval limits (Critical t-value with df = n-1)

```
crit_t <- qt(0.975, df = n_target - 1)

sys_lcl <- sys_distribution_means - (crit_t * sys_distribution_se)
sys_ucl <- sys_distribution_means + (crit_t * sys_distribution_se)

srs_lcl <- srs_distribution_means - (crit_t * srs_distribution_se)
srs_ucl <- srs_distribution_means + (crit_t * srs_distribution_se)
```

Calculate all the metrics using simple base arithmetic

```
# Empirical and Relative Bias
sys_bias  <- mean(sys_distribution_means) - Y_bar_pop
sys_rel_bias <- sys_bias / Y_bar_pop
srs_bias  <- mean(srs_distribution_means) - Y_bar_pop
srs_rel_bias <- srs_bias / Y_bar_pop

# Root Mean Squared Error (RMSE)
sys_rmse  <- sqrt(mean((sys_distribution_means - Y_bar_pop)^2))
srs_rmse  <- sqrt(mean((srs_distribution_means - Y_bar_pop)^2))

# 95% Confidence Interval Coverage Probabilities
sys_coverage <- mean(sys_lcl <= Y_bar_pop & sys_ucl >= Y_bar_pop)
srs_coverage <- mean(srs_lcl <= Y_bar_pop & srs_ucl >= Y_bar_pop)

# Average Confidence Interval Width
sys_avg_wdth <- mean(sys_ucl - sys_lcl)
srs_avg_wdth <- mean(srs_ucl - srs_lcl)

# Monte Carlo Standard Error of the estimated coverage probability
sys_mc_se <- sqrt((sys_coverage * (1 - sys_coverage)) / B)
srs_mc_se <- sqrt((srs_coverage * (1 - srs_coverage)) / B)

# Empirical Design Effect (Deff ratio based on empirical simulation variance)
empirical_deff <- var(sys_distribution_means) / var(srs_distribution_means)


# 4. final score
```

```r
Metrics <- data.frame(
  Metric = c("Empirical Bias", "Relative Bias", "RMSE", "CI Coverage Probability", "Avg CI Width"),
  Systematic = c(sys_bias, sys_rel_bias, sys_rmse, sys_coverage, sys_avg_wdth),
  SRSWOR     = c(srs_bias, srs_rel_bias, srs_rmse, srs_coverage, srs_avg_wdth)
)
```

```r
print(Metrics)
```

```
##                    Metric    Systematic        SRSWOR
## 1          Empirical Bias -4.670342e+00 -2.105486e+01
## 2           Relative Bias -1.818074e-04 -8.196250e-04
## 3                    RMSE  1.670562e+02  6.091339e+02
## 4 CI Coverage Probability  1.000000e+00  9.530000e-01
## 5            Avg CI Width  2.396683e+03  2.391968e+03
```

```r
print(paste("Monte Carlo SE of Systematic Coverage Probability:", round(sys_mc_se, 5)))
```

```
## [1] "Monte Carlo SE of Systematic Coverage Probability: 0"
```

```r
print(paste("Monte Carlo SE of SRSWOR Coverage Probability:", round(srs_mc_se, 5)))
```

```
## [1] "Monte Carlo SE of SRSWOR Coverage Probability: 0.00669"
```

```r
print(paste("Empirical Design Effect (Deff):", round(empirical_deff, 4)))
```

```
## [1] "Empirical Design Effect (Deff): 0.0752"
```

Saving the Population and Samples in CSV files

```r
## Full population

write.csv(pop, file = "full_finite_population.csv", row.names = FALSE)
print("[FILE SAVE COMPLETE] Full finite population successfully exported as 'full_finite_population.csv
```

```
## [1] "[FILE SAVE COMPLETE] Full finite population successfully exported as 'full_finite_population.csv
```

```r
#  probability sample of 1000 households drawn via systematic grid

write.csv(sys_sample, file = "Systematic_sample.csv", row.names = FALSE)
print("[FILE SAVE COMPLETE]  Sample successfully exported as 'Systematic_sample.csv'")
```

```
## [1] "[FILE SAVE COMPLETE]  Sample successfully exported as 'Systematic_sample.csv'"
```

```r
#  probability sample of 1000 households drawn via SRS grid

write.csv(srs_sample, file = "SRS_sample.csv", row.names = FALSE)
print("[FILE SAVE COMPLETE] Sample successfully exported as 'SRS_sample.csv'")
```

```
## [1] "[FILE SAVE COMPLETE] Sample successfully exported as 'SRS_sample.csv'"
```